

broker load manual

broker load manual

Broker Load 通过 MySQL API 发起，由 Doris 根据 LOAD 语句中的信息主动从远程数据源拉取数据。它是一种**异步导入**方式，提交后需要通过 SHOW LOAD 语句查看导入进度与结果。

:::caution Note 该方法已废弃，将在 5.0 版本中移除。请改用语法： "insert into table select xxx from tvf or catalog table". :::

Broker Load 适用于以下典型场景：

- 源数据存储于远程系统（如对象存储、HDFS）
- 单次导入数据量较大（GB 至 TB 级）
- 希望以异步方式批量导入，并由 Doris 自身控制并发与重试

你也可以通过 湖仓一体 / TVF 中的 HDFS TVF 或 S3 TVF 配合 INSERT INTO 实现导入。基于 TVF 的 INSERT INTO 当前为同步导入，而 Broker Load 是异步导入。

在 Doris 早期版本中，S3 Load 和 HDFS Load 都通过 WITH BROKER 连接到具体的 Broker 进程实现。随着版本迭代，S3 Load 与 HDFS Load 已不再依赖额外的 Broker 进程，但仍沿用与 Broker Load 类似的语法。由于历史原因和语法相似性，**S3 Load、HDFS Load 与 Broker Load 三种导入方式被统称为 Broker Load。**

使用限制

下表汇总了 Broker Load 的能力范围：

- 维度：存储后端；支持范围：S3 协议、HDFS 协议、其他协议（需相应 Broker 进程）
- 维度：文件路径模式；支持范围：通配符 *、?、[abc]、[a-z]；范围展开 {1..10}、{a,b,c}。完整语法见文件路径模式
- 维度：数据格式；支持范围：CSV、JSON、PARQUET、ORC
- 维度：压缩类型；支持范围：
PLAIN、GZ、LZO、BZ2、LZ4FRAME、DEFLATE、LZOP、LZ4BLOCK、SNAPP
YBLOCK、ZLIB、ZSTD

基本原理

用户提交导入任务后：

1. FE 生成对应的 Plan，并根据当前 BE 数量与文件大小，将 Plan 分发给多个 BE 执行。
2. 每个 BE 负责导入一部分数据：从 Broker 拉取数据 → 进行数据转换 → 写入 Doris 系统。
3. 所有 BE 完成导入后，由 FE 最终判定导入是否成功。

图片说明：Broker Load 基本原理

BE 通过 Broker 进程读取远程存储系统的数据。引入 Broker 的主要目的是：

- **生态兼容**：用户可使用 Java 按 Broker 标准开发，便于兼容大数据生态中的各类存储系统。
- **错误隔离**：Broker 进程与 BE 进程分离，提升 BE 的稳定性。

当前 BE 已内置对 HDFS 和 S3 的支持，**从 HDFS 或 S3 导入数据无需额外启动 Broker 进程**。如有自定义 Broker 实现，则需部署相应的 Broker 进程。

快速上手

本节以 S3 Load 为例演示完整流程。完整语法请参考 SQL 手册中的 Broker Load。

前置检查

1. Doris 表权限

Broker Load 需要对目标表的 INSERT 权限。如果没有该权限，可通过 GRANT 命令授权。

2. S3 认证和连接信息

以 AWS S3 为例（其他对象存储系统可参考）：

- 信息：AK / SK；获取方式：AWS Console 的 My Security Credentials 中查看或新建 Access keys
- 信息：REGION；获取方式：创建 Bucket 时选择，或在 Bucket 列表中查看
- 信息：ENDPOINT；获取方式：参见 AWS 文档：S3 Endpoints

创建导入作业

步骤 1：准备 S3 上的 CSV 文件

创建 brokerload_example.csv，内容如下：

```
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
9,Emma,37
10,Liam,64
```

步骤 2：在 Doris 中创建目标表

```
CREATE TABLE testdb.test_brokerload(
  user_id    BIGINT    NOT NULL COMMENT "user id",
  name       VARCHAR(20) COMMENT "name",
  age        INT       COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

步骤 3：提交 Broker Load 作业

将 Bucket 名称与 S3 认证信息替换为实际值：

```
LOAD LABEL broker_load_2022_04_01
(
  DATA INFILE("s3://your_bucket_name/brokerload_example.csv")
  INTO TABLE test_brokerload
  COLUMNS TERMINATED BY ","
  FORMAT AS "CSV"
  (user_id, name, age)
)
WITH S3
(
  "provider" = "S3",
  "AWS_ENDPOINT" = "https://s3.us-west-2.amazonaws.com",
```

```

"AWS_ACCESS_KEY" = "",
"AWS_SECRET_KEY"="",
"AWS_REGION" = "us-west-2",
"compress_type" = "PLAIN"
)
PROPERTIES
(
  "timeout" = "3600"
);

```

provider 字段需要根据实际的对象存储服务商填写。Doris 支持的 provider 列表如下：

- provider: S3; 厂商: 亚马逊 AWS
- provider: AZURE; 厂商: 微软 Azure
- provider: GCP; 厂商: 谷歌 GCP
- provider: OSS; 厂商: 阿里云
- provider: COS; 厂商: 腾讯云
- provider: OBS; 厂商: 华为云
- provider: BOS; 厂商: 百度云

如不在列表中（例如 MinIO），可以尝试使用 S3（兼容 AWS 模式）。

查看导入作业

Broker Load 是异步导入，可通过 SHOW LOAD 命令查看具体结果：

```

mysql> show load order by createtime desc limit 1\G;
***** 1. row *****
  JobId: 41326624
  Label: broker_load_2022_04_01
  State: FINISHED
  Progress: ETL:100%; LOAD:100%
  Type: BROKER
  EtlInfo: unselected.rows=0; dpp.abnorm.ALL=0; dpp.norm.ALL=27
  TaskInfo: cluster:N/A; timeout(s):1200; max_filter_ratio:0.1
  ErrorMsg: NULL
  CreateTime: 2022-04-01 18:59:06
  EtlStartTime: 2022-04-01 18:59:11
  EtlFinishTime: 2022-04-01 18:59:11
  LoadStartTime: 2022-04-01 18:59:11
  LoadFinishTime: 2022-04-01 18:59:11

```

URL: **NULL**

```
JobDetails: {"Unfinished backends":{"5072bde59b74b65-8d2c0ee5b029adc0":  
[], "ScannedRows":27, "TaskNumber":1, "All backends":{"5072bde59b74b65-  
8d2c0ee5b029adc0":[36728051]}, "FileName":1, "FileSize":5540}  
1 row in set (0.01 sec)
```

取消导入作业

当 Broker Load 作业状态不是 CANCELLED 或 FINISHED 时，可由用户手动取消。取消时需指定待取消任务的 Label。语法详见 CANCEL LOAD。

例如：取消数据库 demo 上 Label 为 broker_load_2022_04_01 的导入作业：

```
CANCEL LOAD FROM demo WHERE LABEL = "broker_load_2022_04_01";
```

绑定 Compute Group

存算分离模式下，Broker Load 选择 Compute Group 的优先级：

1. 选择 use db@cluster 语句指定的 Compute Group；
2. 选择用户属性 default_compute_group 指定的 Compute Group；
3. 从当前用户有权限的 Compute Group 中选择一个。

存算一体模式下：选择用户属性 resource_tags.location 中指定的 Compute Group；如果用户属性中未指定，则使用名为 default 的 Compute Group。

参考手册

导入命令语法

```
LOAD LABEL load_label  
(  
  data_desc1[, data_desc2, ...]  
  [format_properties]  
)  
WITH [S3|HDFS|BROKER broker_name]  
[broker_properties]  
[load_properties]  
[COMMENT "comments"];
```

WITH 子句指定如何访问存储系统，broker_properties 是该访问方式的配置参数：

- 子句：S3；说明：使用 S3 协议的存储系统
- 子句：HDFS；说明：使用 HDFS 协议的存储系统

- 子句：BROKER broker_name；说明：其他协议的存储系统。可通过 SHOW BROKER 查看可用的 broker_name 列表。详见下文的"其他 Broker 导入"

导入配置参数

导入参数 (*Load Properties*)

- Property 名称：timeout；类型：Long；默认值：14400；说明：导入的超时时间，单位秒。范围 1 ~ 259200 秒
- Property 名称：max_filter_ratio；类型：Float；默认值：0.0；说明：最大可容忍的不规范数据比例，默认零容忍。取值范围 0 ~ 1。错误率超过该值则导入失败。不规范数据不包括通过 WHERE 条件过滤掉的行
- Property 名称：strict_mode；类型：Boolean；默认值：false；说明：是否开启严格模式
- Property 名称：partial_columns；类型：Boolean；默认值：false；说明：是否使用部分列更新，仅在 Unique Key 表 + Merge on Write 时有效
- Property 名称：timezone；类型：String；默认值："Asia/Shanghai"；说明：本次导入使用的时区，会影响所有时区相关函数的结果
- Property 名称：load_parallelism；类型：Integer；默认值：8；说明：每个 BE 上并发 instance 数量的上限
- Property 名称：send_batch_parallelism；类型：Integer；默认值：1；说明：sink 节点发送数据的并发度，仅在关闭 memtable 前移时生效
- Property 名称：load_to_single_tablet；类型：Boolean；默认值：false；说明：是否每个分区只导入一个 tablet。仅允许在使用 random 分桶的 OLAP 表上设置
- Property 名称：priority；类型：HIGH / NORMAL / LOW；默认值：NORMAL；说明：导入任务的优先级

格式参数 (*Format Properties*)

- 参数名：skip_lines；类型：Integer；默认值：0；说明：跳过 CSV 文件开头的若干行。当格式为 csv_with_names 或 csv_with_names_and_types 时无效
- 参数名：trim_double_quotes；类型：Boolean；默认值：false；说明：是否去除字段外层的双引号
- 参数名：enclose；类型：String；默认值：""；说明：字段包含换行符或分隔符时的包裹字符。例如分隔符为 、 包裹字符为 ' 时，'b,c' 会被解析为一个字段
- 参数名：escape；类型：String；默认值：""；说明：用于转义包裹字符的转义字符。例如转义字符为 \ 包裹字符为 '，字段 'b,\c' 将被正确解析为 'b,c'

:::tip 注意：参数应该放在哪里？

- **格式参数**用于定义如何解析源文件（如分隔符、引号处理），应在 LOAD 语句**内部**的 PROPERTIES 中设置。
- **导入参数**用于控制导入行为（如超时、重试），应在 LOAD 语句**外部**最外层的 PROPERTIES 块中设置。 :::

```
LOAD LABEL s3_load_example (
  DATA INFILE("s3://bucket/path/file.csv")
  INTO TABLE users
  COLUMNS TERMINATED BY ","
  FORMAT AS "CSV"
  (user_id, name, age)
  PROPERTIES (
    "trim_double_quotes" = "true" -- 格式参数
  )
)
WITH S3 (
  ...
)
PROPERTIES (
  "timeout" = "3600" -- 导入参数
);
```

fe.conf 系统级配置

下面几个配置属于 Broker Load 的系统级别配置，作用于所有 Broker Load 导入任务。主要通过修改 *fe.conf* 来调整。

- 配置项：min_bytes_per_broker_scanner；类型：Long；默认值：67108864 (64 MB)；说明：单个 BE 处理的数据量的最小值，单位字节
- 配置项：max_bytes_per_broker_scanner；类型：Long；默认值：536870912000 (500 GB)；说明：单个 BE 处理的数据量的最大值，单位字节。一个导入作业支持的最大数据量约为 max_bytes_per_broker_scanner * BE 节点数。需要更大数据量时，应适当调大该值
- 配置项：max_broker_concurrency；类型：Integer；默认值：10；说明：单个作业的最大导入并发数
- 配置项：default_load_parallelism；类型：Integer；默认值：8；说明：每个 BE 节点最大并发 instance 数
- 配置项：broker_load_default_timeout_second；类型：Integer；默认值：14400；说明：Broker Load 导入的默认超时时间，单位秒

导入并发数的计算

最小处理的数据量、最大并发数、源文件大小和当前集群 BE 数量共同决定本次导入的并发数：

本次导入并发数 = $\text{Math.min}(\text{源文件大小} / \text{min_bytes_per_broker_scanner}, \text{max_broker_concurrency}, \text{当前 BE 节点数} * \text{load_parallelism})$

本次导入单个 BE 的处理量 = 源文件大小 / 本次导入的并发数

Session Variable

- Session Variable: time_zone; 类型: String; 默认值: "Asia/Shanghai"; 说明: 默认时区, 会影响导入中时区相关函数的结果
- Session Variable: send_batch_parallelism; 类型: Integer; 默认值: 1; 说明: sink 节点发送数据的并发度, 仅在关闭 memtable 前移时生效

导入示例

下面通过若干典型场景, 展示 Broker Load 的常见用法。

场景 1: 导入 HDFS 上的 TXT 文件

```
LOAD LABEL demo.label_20220402
(
  DATA INFILE("hdfs://host:port/tmp/test_hdfs.txt")
  INTO TABLE `load_hdfs_file_test`
  COLUMNS TERMINATED BY "\t"
  (id,age,name)
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
)
PROPERTIES
(
  "timeout"="1200",
  "max_filter_ratio"="0.1"
);
```

场景 2: HDFS 配置 NameNode HA

```
LOAD LABEL demo.label_20220402
(
  DATA INFILE("hdfs://hafs/tmp/test_hdfs.txt")
  INTO TABLE `load_hdfs_file_test`
```



```

COLUMNS TERMINATED BY "\t"
(id,age,name)
)
with HDFS
(
  "hadoop.username" = "user",
  "fs.defaultFS"="hdfs://hafs",
  "dfs.nameservices" = "hafs",
  "dfs.ha.namenodes.hafs" = "my_namenode1, my_namenode2",
  "dfs.namenode.rpc-address.hafs.my_namenode1" = "nn1_host:rpc_port",
  "dfs.namenode.rpc-address.hafs.my_namenode2" = "nn2_host:rpc_port",
  "dfs.client.failover.proxy.provider.hafs" =
"org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider"
)
PROPERTIES
(
  "timeout"="1200",
  "max_filter_ratio"="0.1"
);

```

场景 3：使用通配符匹配两批文件，分别导入到两张表

Broker Load 支持在文件路径中使用通配符（*、?、[...]）和范围模式（{1..10}）。详细语法请参阅文件路径模式。

```

LOAD LABEL example_db.label2
(
  DATA INFILE("hdfs://host:port/input/file-10*")
  INTO TABLE `my_table1`
  PARTITION (p1)
  COLUMNS TERMINATED BY ","
  (k1, tmp_k2, tmp_k3)
  SET (
    k2 = tmp_k2 + 1,
    k3 = tmp_k3 + 1
  ),
  DATA INFILE("hdfs://host:port/input/file-20*")
  INTO TABLE `my_table2`
  COLUMNS TERMINATED BY ","
  (k1, k2, k3)
)

```

with HDFS

```
(  
  "fs.defaultFS"="hdfs://host:port",  
  "hadoop.username" = "user"  
);
```

使用通配符匹配 file-10* 与 file-20* 两批文件，分别导入到 my_table1 与 my_table2。其中 my_table1 指定导入到分区 p1，并将源文件中第二、三列的值 +1 后导入。

场景 4：使用通配符从 HDFS 导入一批数据

LOAD **LABEL** example_db.label3

```
(  
  DATA INFILE("hdfs://host:port/user/doris/data/*/*")  
  INTO TABLE `my_table`  
  COLUMNS TERMINATED BY "\\x01"  
)
```

with HDFS

```
(  
  "fs.defaultFS"="hdfs://host:port",  
  "hadoop.username" = "user"  
);
```

指定分隔符为 Hive 常用的默认分隔符 \\x01，并使用通配符 * 指定 data 目录下所有子目录中的所有文件。

场景 5：导入 Parquet 格式数据

LOAD **LABEL** example_db.label4

```
(  
  DATA INFILE("hdfs://host:port/input/file")  
  INTO TABLE `my_table`  
  FORMAT AS "parquet"  
  (k1, k2, k3)  
)
```

with HDFS

```
(  
  "fs.defaultFS"="hdfs://host:port",  
  "hadoop.username" = "user"  
);
```

如未指定 FORMAT AS，默认通过文件后缀判断格式。

场景 6：从文件路径中提取分区字段

```
LOAD LABEL example_db.label5
(
  DATA INFILE("hdfs://host:port/input/city=beijing/*/*")
  INTO TABLE `my_table`
  FORMAT AS "csv"
  (k1, k2, k3)
  COLUMNS FROM PATH AS (city, utc_date)
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

my_table 表中的列为 k1, k2, k3, city, utc_date。

hdfs://hdfs_host:hdfs_port/user/doris/data/input/dir/city=beijing 目录下包含如下文件：

```
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-01/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=beijing/utc_date=2020-10-02/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-03/0000.csv
hdfs://hdfs_host:hdfs_port/input/city=tianji/utc_date=2020-10-04/0000.csv
```

文件中只包含 k1, k2, k3 三列数据，city, utc_date 这两列从文件路径中提取。

场景 7：对导入数据进行过滤

```
LOAD LABEL example_db.label6
(
  DATA INFILE("hdfs://host:port/input/file")
  INTO TABLE `my_table`
  (k1, k2, k3)
  SET (
    k2 = k2 + 1
  )
  PRECEDING FILTER k1 = 1
  WHERE k1 > k2
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
```

```
"hadoop.username" = "user"
);
```

只有原始数据中 $k1 = 1$ ，并且转换后 $k1 > k2$ 的行才会被导入。

场景 8：从文件路径中提取时间分区字段

```
LOAD LABEL example_db.label7
(
  DATA INFILE("hdfs://host:port/user/data/*/test.txt")
  INTO TABLE `tbl12`
  COLUMNS TERMINATED BY ","
  (k2,k3)
  COLUMNS FROM PATH AS (data_time)
  SET (
    data_time=str_to_date(data_time, '%Y-%m-%d %H%%3A%i%%3A%s')
  )
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username" = "user"
);
```

:::tip 时间包含 %3A。HDFS 路径中不允许包含 :，所有 : 会被 %3A 替换。 :::

路径下有如下文件：

```
/user/data/data_time=2020-02-17 00%3A00%3A00/test.txt
/user/data/data_time=2020-02-18 00%3A00%3A00/test.txt
```

表结构：

```
CREATE TABLE IF NOT EXISTS tbl12 (
  data_time DATETIME,
  k2      INT,
  k3      INT
) DISTRIBUTED BY HASH(data_time) BUCKETS 10
PROPERTIES (
  "replication_num" = "3"
);
```

场景 9：使用 Merge 方式导入

```
LOAD LABEL example_db.label8
(
  MERGE DATA INFILE("hdfs://host:port/input/file")
  INTO TABLE `my_table`
  (k1, k2, k3, v2, v1)
  DELETE ON v2 > 100
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
)
PROPERTIES
(
  "timeout" = "3600",
  "max_filter_ratio" = "0.1"
);
```

my_table 必须为 Unique Key 表。当导入数据中 $v2 > 100$ 时，该行被视为删除行。导入超时 3600 秒，允许错误率 10%。

场景 10：指定 source_sequence 列保证替换顺序

```
LOAD LABEL example_db.label9
(
  DATA INFILE("hdfs://host:port/input/file")
  INTO TABLE `my_table`
  COLUMNS TERMINATED BY ","
  (k1,k2,source_sequence,v1,v2)
  ORDER BY source_sequence
)
with HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

my_table 必须为 Unique Key 模型表，并指定了 Sequence 列。数据将按源数据中 source_sequence 列的值保证顺序性。

场景 11：导入 JSON，并指定 json_root / jsonpaths

```
LOAD LABEL example_db.label10
(
  DATA INFILE("hdfs://host:port/input/file.json")
  INTO TABLE `my_table`
  FORMAT AS "json"
  PROPERTIES(
    "json_root" = "$.item",
    "jsonpaths" = "["$.id", "$.city", "$.code"]"
  )
)
WITH HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

jsonpaths 也可以与 column list 及 SET (column_mapping) 配合使用：

```
LOAD LABEL example_db.label10
(
  DATA INFILE("hdfs://host:port/input/file.json")
  INTO TABLE `my_table`
  FORMAT AS "json"
  (id, code, city)
  SET (id = id * 10)
  PROPERTIES(
    "json_root" = "$.item",
    "jsonpaths" = "["$.id", "$.city", "$.code"]"
  )
)
WITH HDFS
(
  "fs.defaultFS"="hdfs://host:port",
  "hadoop.username"="user"
);
```

:::info 备注 如果要将 JSON 文件中根节点的 JSON 对象导入，jsonpaths 需指定为 \$。，即 PROPERTIES("jsonpaths"="\$.")。 :::

高级配置

S3 Load URL 访问方式

S3 SDK 默认使用 virtual-hosted-style 访问。但某些对象存储系统未开启或不支持 virtual-hosted-style，可以添加 use_path_style 参数强制使用 path-style：

WITH S3

```
(  
  "AWS_ENDPOINT" = "AWS_ENDPOINT",  
  "AWS_ACCESS_KEY" = "AWS_ACCESS_KEY",  
  "AWS_SECRET_KEY" = "AWS_SECRET_KEY",  
  "AWS_REGION" = "AWS_REGION",  
  "use_path_style" = "true"  
)
```

S3 Load 临时密钥

支持使用临时密钥（TOKEN）访问所有支持 S3 协议的对象存储：

WITH S3

```
(  
  "AWS_ENDPOINT" = "AWS_ENDPOINT",  
  "AWS_ACCESS_KEY" = "AWS_TEMP_ACCESS_KEY",  
  "AWS_SECRET_KEY" = "AWS_TEMP_SECRET_KEY",  
  "AWS_TOKEN" = "AWS_TEMP_TOKEN",  
  "AWS_REGION" = "AWS_REGION"  
)
```

HDFS 认证方式

1. 简单认证

简单认证即 Hadoop 配置 hadoop.security.authentication 为 simple：

```
(  
  "username" = "user",  
  "password" = ""  
);
```

username 配置为要访问的用户，密码置空即可。

2. Kerberos 认证

该认证方式需提供以下信息：

- 参数：hadoop.security.authentication；说明：指定认证方式为 kerberos
- 参数：hadoop.kerberos.principal；说明：指定 Kerberos 的 principal
- 参数：hadoop.kerberos.keytab；说明：指定 Kerberos 的 keytab 文件路径。该文件必须为 Broker 进程所在服务器上的绝对路径，且 Broker 进程可访问
- 参数：kerberos_keytab_content；说明：指定 keytab 文件内容经 base64 编码后的字符串，与 hadoop.kerberos.keytab 二选一

示例：

```
(  
  "hadoop.security.authentication" = "kerberos",  
  "hadoop.kerberos.principal" = "doris@YOUR.COM",  
  "hadoop.kerberos.keytab" = "/home/doris/my.keytab"  
)  
(  
  "hadoop.security.authentication" = "kerberos",  
  "hadoop.kerberos.principal" = "doris@YOUR.COM",  
  "kerberos_keytab_content" = "ASDOWHDLAWIDJHWLDKSALDJSDIWALD"  
)
```

采用 Kerberos 认证方式，需要 krb5.conf 文件。该文件包含 Kerberos 配置信息，通常应安装在 /etc 目录下，也可通过环境变量 KRB5_CONFIG 覆盖默认位置。krb5.conf 内容示例：

```
[libdefaults]  
  default_realm = DORIS.HADOOP  
  default_tkt_enctypes = des3-hmac-sha1 des-cbc-crc  
  default_tgs_enctypes = des3-hmac-sha1 des-cbc-crc  
  dns_lookup_kdc = true  
  dns_lookup_realm = false  
  
[realms]  
  DORIS.HADOOP = {  
    kdc = kerberos-doris.hadoop.service:7005  
  }
```


HDFS HA 模式

该配置用于访问以 HA 模式部署的 HDFS 集群。

- 参数：dfs.nameservices；说明：指定 HDFS 服务的名字（自定义），如 "dfs.nameservices" = "my_ha"
- 参数：dfs.ha.namenodes.xxx；说明：自定义 NameNode 名字（多个用逗号分隔）。xxx 为 dfs.nameservices 的自定义名字，如 "dfs.ha.namenodes.my_ha" = "my_nn"
- 参数：dfs.namenode.rpc-address.xxx.nn；说明：指定 NameNode 的 RPC 地址。nn 是 dfs.ha.namenodes.xxx 中的 NameNode 名字，如 "dfs.namenode.rpc-address.my_ha.my_nn" = "host:port"
- 参数：dfs.client.failover.proxy.provider.[nameservice ID]；说明：指定 Client 连接 NameNode 的 provider，默认为 org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider

示例：

```
(  
  "fs.defaultFS" = "hdfs://my_ha",  
  "dfs.nameservices" = "my_ha",  
  "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",  
  "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",  
  "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",  
  "dfs.client.failover.proxy.provider.my_ha" =  
  "org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider"  
)
```

HA 模式可与上述两种认证方式组合使用。例如通过简单认证访问 HA HDFS：

```
(  
  "username"="user",  
  "password"="passwd",  
  "fs.defaultFS" = "hdfs://my_ha",  
  "dfs.nameservices" = "my_ha",  
  "dfs.ha.namenodes.my_ha" = "my_namenode1, my_namenode2",  
  "dfs.namenode.rpc-address.my_ha.my_namenode1" = "nn1_host:rpc_port",  
  "dfs.namenode.rpc-address.my_ha.my_namenode2" = "nn2_host:rpc_port",  
  "dfs.client.failover.proxy.provider.my_ha" =  
  "org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvider"  
)
```

其他 Broker 导入

其他远端存储系统的 Broker 是 Doris 集群中的可选进程，主要用于支持 Doris 对远端存储中文件和目录的读写。目前 Doris 提供了多种远端存储系统的 Broker 实现：

- 腾讯云 CHDFS
- 腾讯云 GFS
- JuiceFS

历史版本中 Doris 还支持过各类对象存储的 Broker，但当前**更推荐使用 WITH S3 方式**导入对象存储数据，而不再推荐 WITH BROKER。

Broker 通过提供 RPC 服务端口对外服务，是一个无状态的 Java 进程，负责为远端存储的读写操作封装类 POSIX 的文件操作（如 open、pread、pwrite 等）。Broker 不记录任何其他信息，包括远端存储的连接信息、文件信息、权限信息等，都需通过 RPC 调用中的参数传入。

Broker 仅作为数据通路，并不参与计算，因此内存占用较少。通常一个 Doris 系统中会部署一个或多个 Broker 进程。相同类型的 Broker 会组成一个组，并设定一个名称（Broker name）。

Broker 信息

Broker 信息包括 **名称** 和 **认证信息** 两部分，常用语法如下：

```
WITH BROKER "broker_name"
(
  "username" = "xxx",
  "password" = "yyy",
  "other_prop" = "prop_value",
  ...
);
```

名称（Broker Name）

通过 WITH BROKER "broker_name" 子句指定一个已存在的 Broker Name。Broker Name 是用户在通过 ALTER SYSTEM ADD BROKER 命令添加 Broker 进程时指定的名称。一个名称通常对应一个或多个 Broker 进程，Doris 会根据名称选择可用的 Broker 进程。可通过 SHOW BROKER 查看集群中已存在的 Broker。

:::info 备注 Broker Name 只是一个用户自定义名称，不代表 Broker 的类型。 :::

认证信息

不同 Broker 类型与不同的访问方式需要提供不同的认证信息。认证信息通常在 WITH BROKER "broker_name" 之后的 Property Map 中以 Key-Value 方式提供。

各类 *Broker* 的连接配置

阿里云 OSS

```
(  
  "fs.oss.accessKeyId" = "",  
  "fs.oss.accessKeySecret" = "",  
  "fs.oss.endpoint" = ""  
)
```

百度云 BOS

使用 BOS 时需下载相应的 SDK 包，具体配置与使用可参考 BOS HDFS 官方文档。下载并解压后，将 jar 包放到 Broker 的 lib 目录下。

```
(  
  "fs.bos.access.key" = "xx",  
  "fs.bos.secret.access.key" = "xx",  
  "fs.bos.endpoint" = "xx"  
)
```

华为云 OBS

```
(  
  "fs.obs.access.key" = "xx",  
  "fs.obs.secret.key" = "xx",  
  "fs.obs.endpoint" = "xx"  
)
```

JuiceFS

```
(  
  "fs.defaultFS" = "jfs://xxx/",  
  "fs.jfs.impl" = "io.juicefs.JuiceFileSystem",  
  "fs.AbstractFileSystem.jfs.impl" = "io.juicefs.JuiceFS",  
  "juicefs.meta" = "xxx",  
  "juicefs.access-log" = "xxx"  
)
```

GCS

使用 Broker 访问 GCS 时, Project ID 是必须的, 其他参数可选, 所有参数配置请参见 GCS Config:

```
(  
  "fs.gs.project.id" = "Your Project ID",  
  "fs.AbstractFileSystem.gs.impl" = "com.google.cloud.hadoop.fs.gcs.GoogleHadoopFS",  
  "fs.gs.impl" = "com.google.cloud.hadoop.fs.gcs.GoogleHadoopFileSystem",  
)
```

常见问题与故障排查

常见报错

1. 导入报错: Scan bytes per broker scanner exceed limit:xxx

请参考"导入超时"章节, 修改 FE 配置项 max_bytes_per_broker_scanner 与 max_broker_concurrency。

2. 导入报错: failed to send batch 或 TabletWriter add batch with unknown id

适当调整 query_timeout 与 streaming_load_rpc_max_alive_time_sec。

3. 导入报错: LOAD_RUN_FAIL; msg:Invalid Column Name:xxx

如果是 PARQUET 或 ORC 格式的数据, 文件头中的列名需要与 Doris 表中的列名保持一致。例如:

```
(tmp_c1,tmp_c2)  
SET  
(  
  id=tmp_c2,  
  name=tmp_c1  
)
```

含义为: 从 Parquet 或 ORC 文件中获取列名为 (tmp_c1, tmp_c2) 的列, 映射到 Doris 表的 (id, name) 列。如果未设置 SET, 则以 column 中的列作为映射。

注意: 某些 Hive 版本直接生成的 ORC 文件, 文件中的表头并非 Hive Meta 数据, 而是 (_col0, _col1, _col2, ...), 可能导致 Invalid Column Name 错误, 此时需要使用 SET 进行映射。

4. 导入报错: Failed to get S3 FileSystem for bucket is null/empty

Bucket 信息填写不正确或不存在，或 Bucket 格式不受支持。例如使用 GCS 创建带 _ 的桶名时（如 s3://gs_bucket/load_tbl），S3 Client 访问 GCS 会报错；建议创建 Bucket 路径时不使用 _。

5. 导入超时

导入的 timeout 默认超时时间为 4 小时。如果超时，**不推荐**直接将最大超时时间调大解决。单次导入超过 4 小时时，建议通过切分待导入文件并分多次导入来解决，因为超时时间设置过大会导致单次失败后重试的时间成本很高。

可通过如下公式估算 Doris 集群期望的最大单次导入文件数据量：

期望最大导入文件数据量 = $14400s * 10M/s * BE \text{ 个数}$

例如：集群 BE 个数为 10

期望最大导入文件数据量 = $14400s * 10M/s * 10 = 1440000M \approx 1440G$

注意：一般用户环境可能达不到 10M/s 的速度，所以建议超过 500G 的文件都进行切分后再导入。

更多帮助

关于 Broker Load 的更多详细语法及最佳实践，请参阅 Broker Load 命令手册。也可以在 MySQL 客户端命令行中执行 HELP BROKER LOAD 获取更多帮助信息。